BCSE307P — Compiler Design Lab

Assessment 9 — Project Review II: Progress Report

Kelvin

An AI-Assisted Domain-Specific Language Compiler for IoT Automation

Submitted by

R. Sanjay Reg. No.: _____

Settipalli Tejovardhan Reg. No.: _____

Ashutosh Reg. No.: _____

Under the guidance of

Faculty: _____

Vellore Institute of Technology

October 2026

Abstract

Home and building automation is usually expressed as trigger–action rules, but the people who need these rules are rarely programmers, and natural-language assistants based on large language models can silently produce rules that are wrong. This project presents Kelvin, an AI-assisted compiler for a small domain-specific language (DSL) for IoT automation. A user describes a rule in English or by voice; a free language model (Ollama, Groq or Gemini, with an offline rule-based fallback) drafts DSL code, and a hand-written C++17 compiler verifies it before it can be used. The compiler implements all seven classical phases: a DFA-based lexer, an LL(1) recursive-descent parser, an abstract syntax tree, semantic analysis with satisfiability-based rule-conflict detection, three-address intermediate code, a control-flow-graph optimizer with liveness and available-expressions analysis, and a retargetable back end for C, Arduino, Python and JavaScript. A web studio visualises every phase and runs the optimized code in a smart-home digital twin. On 13 example programs the compiler finishes in under 0.6 ms, the optimizer removes up to 52% of the intermediate code, and all 145 automated tests pass, including equivalence checks across the generated C, Python and JavaScript programs.

Keywords: compiler design, domain-specific language, IoT automation, LL(1) parsing, rule-conflict analysis, dataflow optimization, code generation, large language models, digital twin.

Contents

Abstract 2
1. Introduction 4
  1.1 Motivation 4
  1.2 Problem Statement 4
  1.3 Objectives 4
  1.4 Scope of the Language and Compiler 5
  1.5 Organisation of the Report 5
2. Related Work 6
  2.1 Compiler Construction and Parsing 6
  2.2 Program Analysis and Optimization 6
  2.3 Domain-Specific Languages 6
  2.4 Trigger–Action Programming and IoT Safety 6
  2.5 Natural Language to Code 6
  2.6 Research Gap 8
3. Proposed Methodology 9
  3.1 Architecture 9
  3.2 Language Design 9
  3.3 Compiler Phases 10
  3.4 AI Assistant and Verification Gate 11
  3.5 Web Studio and Digital Twin 11
  3.6 Tools and Technologies 12
4. Results and Discussion 13
  4.1 Test Cases 13
  4.2 Sample Outputs 14
  4.3 Error Handling 17
  4.4 Performance 18
  4.5 Novelty 20
  4.6 Limitations 20
5. Conclusion and Future Work 22
References 23

(To refresh page numbers after editing, right-click this list in Word and choose Update Field.)

1. Introduction

1.1 Motivation

Smart homes, greenhouses and offices are controlled by small automation rules of the form *"if a sensor reads something, switch a device"*. Studies of trigger–action platforms show that such rules are written in very large numbers by end users who are not programmers [15], [16], and that these users often misunderstand how their rules behave [17], [18]. Natural-language assistants based on large language models (LLMs) [26], [27] make it easy to *write* rules, but they do not guarantee that a rule is *correct*: a model can drop a condition, use an undeclared device, or produce two rules that fight over the same device. In the physical world these mistakes switch off heaters, leave doors unlocked or flood gardens [19], [21].

Compiler technology is the natural answer to this problem. A precisely defined language, a parser and a type checker can reject malformed rules; static analysis can find rules that can never fire or that conflict; and an optimizing back end can turn verified rules into small, predictable code for microcontrollers. This project combines both worlds: the language model is used only as a *front-end convenience*, and a complete, hand-written compiler is the authority on what is correct.

1.2 Problem Statement

Design and implement an AI-assisted compiler for a domain-specific language for IoT automation. The system must convert a natural-language rule (for example, *"If the temperature is above 30 and the door is open, turn on the fan"*) into DSL code such as `IF temperature > 30 AND door == OPEN THEN turn_on(fan)`, and compile that code through lexical analysis, parsing, semantic analysis, intermediate representation, optimization and code generation, producing executable code and clear diagnostics.

1.3 Objectives

1. Define a small, unambiguous rule language for sensors, devices and actions, with an LL(1) grammar.
2. Implement all seven compiler phases by hand in C++17, without parser generators.
3. Go beyond type checking with **rule analysis**: detect impossible, always-true and unreachable conditions and conflicting rules, with example inputs.
4. Build an optimizer on basic blocks and a control-flow graph (CFG) using dataflow analysis.
5. Generate code for several targets (C, Arduino/ESP32, Python, JavaScript) from one intermediate representation.
6. Use free language models to convert English to DSL, and **verify every AI answer with the compiler** before it is used.

7. Provide a web studio that visualises every phase and simulates the compiled rules in a smart-home digital twin.

1.4 Scope of the Language and Compiler

In scope. The language supports numeric sensors (`NUMBER`) and state sensors (`STATE`: ON, OFF, OPEN, CLOSED, TRUE, FALSE); device declarations; rules with `IF … THEN`, any number of `ELSE IF` branches and an optional `ELSE`; conditions built from comparisons, `AND`, `OR`, `NOT`, parentheses and arithmetic (`+ − × ÷`, unary minus); and four built-in actions: `turn_on`, `turn_off`, `toggle` and `set_level`. Rules are evaluated on a consistent snapshot of sensor values, as in a programmable-logic-controller scan cycle.

Out of scope. Timers and schedules, user-defined functions, loops, variables that persist between evaluations, and direct device drivers. The Arduino target produces pin tables and stub sensor reads that must be connected to real hardware by the user.

1.5 Organisation of the Report

Section 2 reviews related work in compiler construction, domain-specific languages, trigger–action programming, IoT safety and natural-language code generation. Section 3 presents the proposed methodology, the architecture and the tools. Section 4 reports test cases, sample outputs, error handling, performance numbers and limitations. Section 5 concludes and outlines future work.

2. Related Work

This section reviews 30 works in five groups. Table 1 at the end summarises each work and how it relates to this project.

2.1 Compiler Construction and Parsing

The classical compiler pipeline used in this project follows Aho, Lam, Sethi and Ullman [1], which defines the lexical, syntactic, semantic, intermediate-code, optimization and code-generation phases, and the engineering treatment of Cooper and Torczon [2] and Appel [3]. Muchnick [4] describes the optimization passes (constant folding, common sub-expression elimination, copy propagation and dead-code elimination) that our optimizer implements. Knuth's LR theory [5] and the LL-family parser generators ANTLR [6] and LL(*) [7] represent the generator-based approach; we deliberately wrote a recursive-descent parser by hand and instead had the compiler compute FIRST, FOLLOW and the LL(1) table of its own grammar to prove it is conflict-free.

2.2 Program Analysis and Optimization

Allen [8] introduced control-flow analysis on basic blocks, and Kildall [9] gave the lattice framework for iterative dataflow analysis that we use for liveness and available expressions. Cytron *et al.* [10] introduced static single assignment form, and LLVM [11] showed the value of one intermediate representation shared by many back ends. Our three-address code plays the same role at a much smaller scale: one optimized IR feeds four code generators and the browser's virtual machine.

2.3 Domain-Specific Languages

Mernik, Heering and Sloane [12] give the standard guidance on when and how to build a DSL, van Deursen, Klint and Visser [13] survey DSL design, and Fowler [14] describes external DSLs with their own parser, which is the approach taken here. Our language is an external DSL whose notation is close to the trigger–action rules end users already write.

2.4 Trigger–Action Programming and IoT Safety

Ur *et al.* showed that end users can write trigger–action rules for smart homes [15] and analysed 200,000 IFTTT recipes [16]. Huang and Cakmak [17] and Brackenbury *et al.* [18] found that users frequently misinterpret how such rules behave and introduce bugs. On the safety side, Soteria [19] model-checks IoT apps for unsafe states, IoTGuard [20] enforces policies at run time, Ding and Hu [21] study unsafe physical interactions between devices, and Wang *et al.* [22] chart the attack surface of trigger–action platforms. These works motivate our rule analysis, which detects conflicting and unreachable rules **at compile time** and explains each warning with a concrete example input.

2.5 Natural Language to Code

Desai *et al.* [23] synthesise DSL programs from natural language, NL2Bash [24] maps English to shell commands, and Yin and Neubig [25] generate code with a syntax-directed neural model. Large language models [26], [27], [28] made natural-language programming widely available but also showed that generated code is often subtly wrong. Synchromesh [29] constrains decoding with a grammar to improve reliability. Our system uses a small local model (Qwen3 [30]) and adds a compiler-based verification gate: every answer is compiled, failed answers receive one repair attempt with the compiler's own error messages, and a meaning check rejects answers that drop numbers or conditions from the sentence.

Table 1: *Summary of related work and its relation to this project*

Ref.	Work	Key idea	Use in / gap addressed by this project
[1]	Aho et al., 2006	Seven-phase compiler model	Structure of our compiler
[2]	Cooper & Torczon, 2011	Compiler engineering	IR and optimizer design
[3]	Appel, 1998	Compiler implementation in C	Hand-written front end
[4]	Muchnick, 1997	Optimization catalogue	Our eight optimization passes
[5]	Knuth, 1965	LR parsing theory	Grammar analysis background
[6]	Parr & Quong, 1995	ANTLR LL(k) generator	We avoid generators; prove LL(1) instead
[7]	Parr & Fisher, 2011	LL(*) parsing	Contrast with our LL(1) grammar
[8]	Allen, 1970	Control-flow analysis	Basic blocks and CFG
[9]	Kildall, 1973	Iterative dataflow	Liveness, available expressions
[10]	Cytron et al., 1991	SSA form	Future work (SSA IR)
[11]	Lattner & Adve, 2004	One IR, many back ends	One TAC IR for four targets
[12]	Mernik et al., 2005	When to build a DSL	Language design decisions
[13]	van Deursen et al., 2000	DSL survey	Notation close to the domain
[14]	Fowler, 2010	External DSLs	Own lexer and parser
[15]	Ur et al., 2014	End-user TAP in homes	Target users and rule shape
[16]	Ur et al., 2016	200k IFTTT recipes	Realistic rule patterns
[17]	Huang & Cakmak, 2015	Users' mental models	Need for clear diagnostics
[18]	Brackenbury et al., 2019	Bugs in TAP rules	Conflict and reachability warnings
[19]	Celik et al., 2018 (Soteria)	Model checking IoT apps	Static analysis at compile time
[20]	Celik et al., 2019 (IoTGuard)	Run-time policy enforcement	We check before deployment
[21]	Ding & Hu, 2018	Unsafe device interactions	Device conflict detection
[22]	Wang et al., 2019	TAP attack surface	Local-only design, phone remote key
[23]	Desai et al., 2016	NL to DSL synthesis	English-to-DSL step
[24]	Lin et al., 2018 (NL2Bash)	NL to commands	Natural-language interface
[25]	Yin & Neubig, 2017	Syntax-directed generation	Grammar-aware generation
[26]	Brown et al., 2020	Few-shot LLMs	Prompt with examples
[27]	Chen et al., 2021	LLMs for code	Generated code can be wrong
[28]	Austin et al., 2021	LLM program synthesis	Repair with feedback
[29]	Poesia et al., 2022	Constrained decoding	We verify with a full compiler

Ref.	Work	Key idea	Use in / gap addressed by this project
[30]	Yang et al., 2025 (Qwen3)	Open LLM	Local model used in the system

2.6 Research Gap

Existing trigger–action platforms accept rules without compiler-level checking; safety analysers work on finished apps rather than on a language with a grammar; and natural-language code generators trust the model's output. No surveyed system combines (a) a small DSL with a provably LL(1) grammar, (b) compile-time detection of conflicting and unreachable rules, (c) an optimizing, retargetable back end for microcontrollers and scripting languages, and (d) an LLM front end whose every answer is verified by the compiler. Kelvin addresses this gap.

3. Proposed Methodology

3.1 Architecture

Figure 1 shows the architecture. An English sentence, typed or spoken, is converted to DSL by the AI assistant. The result passes through a **verification gate**: it is compiled by `iotc`, and a meaning check confirms that every number in the sentence appears in the program and every declared sensor is used. The DSL program then flows through the front end (lexer, parser, AST, semantic and rule analysis), the middle end (three-address IR, CFG and dataflow analysis, optimizer) and the back end (code generators and runtimes). A Python web server exposes the compiler to the browser studio, which visualises every phase, and to a digital twin that runs the optimized IR.

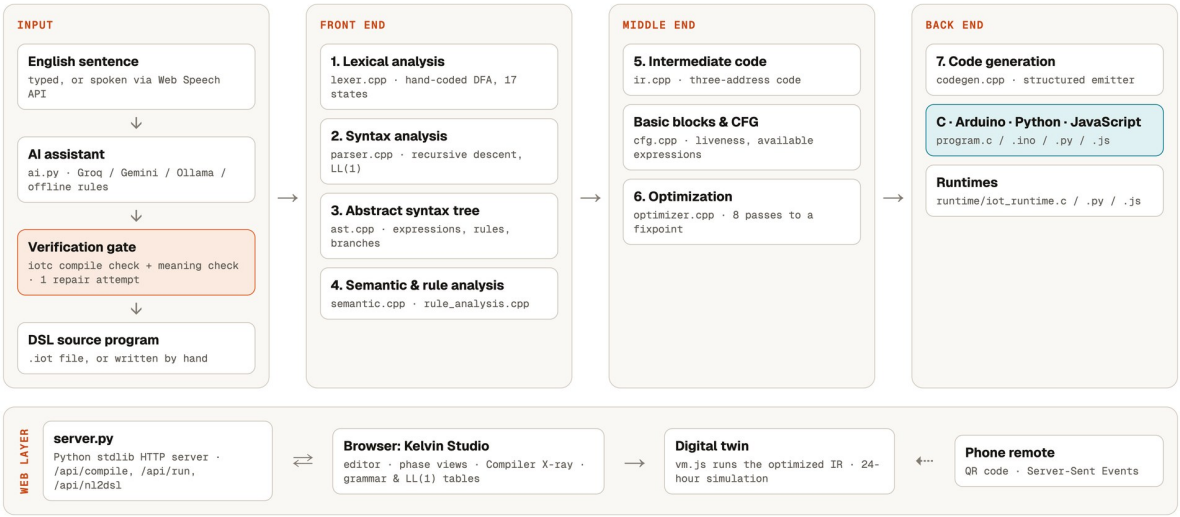


Figure 1: *System architecture of Kelvin*

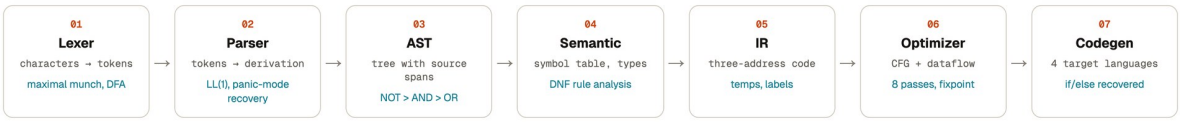


Figure 2: *The seven compiler phases and what each produces*

3.2 Language Design

The grammar below is written in EBNF. The compiler stores the equivalent BNF grammar (61 productions, 29 nonterminals, 33 terminals) as data, computes FIRST and FOLLOW sets and the LL(1) parse table, and reports **0 conflicts**, which proves the grammar is LL(1). Operator precedence, from highest to lowest, is unary minus, `* /`, `+ -`, comparison, `NOT`, `AND`, `OR`.

```
program     → { declaration | rule } EOF
declaration → SENSOR IDENT ':' ( NUMBER | STATE )
            | DEVICE IDENT { ',' IDENT }
rule        → IF expr THEN actions { ELSE IF expr THEN actions } [ ELSE actions ]
actions     → action { action }
action      → IDENT '(' [ expr { ',' expr } ] ')'
expr        → and_expr { OR and_expr }
```

```
and_expr    → not_expr { AND not_expr }
not_expr    → NOT not_expr | rel_expr
rel_expr    → arith [ relop arith ]
arith       → term { ( '+' | '-' ) term }
term        → unary { ( '*' | '/' ) unary }
unary       → '-' unary | primary
primary     → NUMBER_LITERAL | IDENT | ON | OFF | OPEN | CLOSED | TRUE | FALSE | '('
expr ')'
relop       → '>' | '<' | '>=' | '<=' | '==' | '!='
```

3.3 Compiler Phases

Phase 1 — Lexical analysis

The scanner is a hand-coded deterministic finite automaton with 17 states that reads one character at a time and applies the longest-match (maximal munch) rule. Keywords are recognised after an identifier is scanned. Every token records its line and column. Lexical errors, such as an unknown character or = used instead of ==, are reported with a suggested fix, and scanning continues.

Phases 2–3 — Syntax analysis and the abstract syntax tree

A recursive-descent parser implements one C++ function per nonterminal (`parseRule`, `parseExpr`, `parseAnd`, `parseRelational`, …). One token of lookahead always decides the production. After a syntax error, panic-mode recovery skips to the next `IF`, `SENSOR` or `DEVICE`, so several errors are reported in one run. The AST uses a single expression hierarchy (binary, unary and operand nodes) for both conditions and arithmetic, and every node keeps its source span for error messages and cross-linking in the studio.

Phase 4 — Semantic and rule analysis

The semantic analyser builds a symbol table, checks that each name is declared exactly once, infers types (NUMBER, STATE, BOOL) and checks comparisons, arithmetic and action arguments. Misspelt names produce *did-you-mean* suggestions using edit distance, and many errors carry machine-applicable quick fixes. **Rule analysis** then converts each condition to disjunctive normal form and reasons about each conjunction with numeric intervals and sets of states. It reports conditions that can never be true, conditions that are always true, `ELSE IF` branches made unreachable by earlier branches, and pairs of rules that drive the same device to different states for the same inputs, with an example input that triggers the conflict.

Phase 5 — Intermediate code

The IR is three-address code with temporaries (`t1, t2, …`), labels and the instructions `IF_FALSE … GOTO`, `GOTO` and `CALL`. Short `ELSE IF` chains become label sequences. Each instruction stores the source span it came from.

Phase 6 — Optimization

The IR is split into basic blocks and a control-flow graph. Two iterative dataflow analyses are computed [9]: **liveness** (backward) and **available expressions** (forward). Eight passes run repeatedly until the code stops changing: constant folding, algebraic simplification, common

sub-expression elimination (using available expressions), copy propagation, constant-branch elimination, unreachable-code elimination (CFG reachability), jump optimization and dead-code elimination (using liveness). Every intermediate state is recorded, so the studio can replay the optimizer pass by pass.

Phase 7 — Code generation

A structured emitter recovers `if / else` blocks from the optimized IR and drives four target back ends through one interface: **C** (linked with a small runtime), **Arduino/ESP32** (`setup()` and `loop()` with a pin table), **Python** and **JavaScript**, each with its own runtime that prints identical output. The compiler can also translate a program back into plain English.

3.4 AI Assistant and Verification Gate

The AI module uses only free providers, in this order of configuration: Groq, Google Gemini, a local Ollama model (Qwen3 8B in our setup), and a built-in rule-based converter when no model is available. The prompt contains the language reference and the names already declared in the editor. The model's answer is cleaned and compiled. If it has errors, the compiler's error messages are sent back for one repair attempt. A **meaning check** then rejects answers that compile but lose part of the sentence; we added it after observing the local model translate *"above 30 and the door is open"* as `> 3`. If the repaired answer still fails, the rule-based converter is used. The same gate protects the *AI fix* feature, which is offered only when the targeted error disappears and no new error appears.

3.5 Web Studio and Digital Twin

The studio shows the source editor (live compilation, autocomplete, hover types and quick fixes), a seven-phase status strip and tabs for tokens, the AST graph, the grammar with FIRST/FOLLOW sets and the LL(1) table, symbols, IR (original, optimized, optimizer time machine and CFG), generated code with *Build & run*, and problems. The **Compiler X-ray** replays a compilation from traces recorded by the compiler (DFA steps, parser calls and optimizer snapshots). The **digital twin** executes the optimized IR in a small virtual machine and animates devices on a floor plan; a 24-hour simulation drives sensors along daily curves with simple physics, and a phone can act as a sensor panel through a QR code and Server-Sent Events.

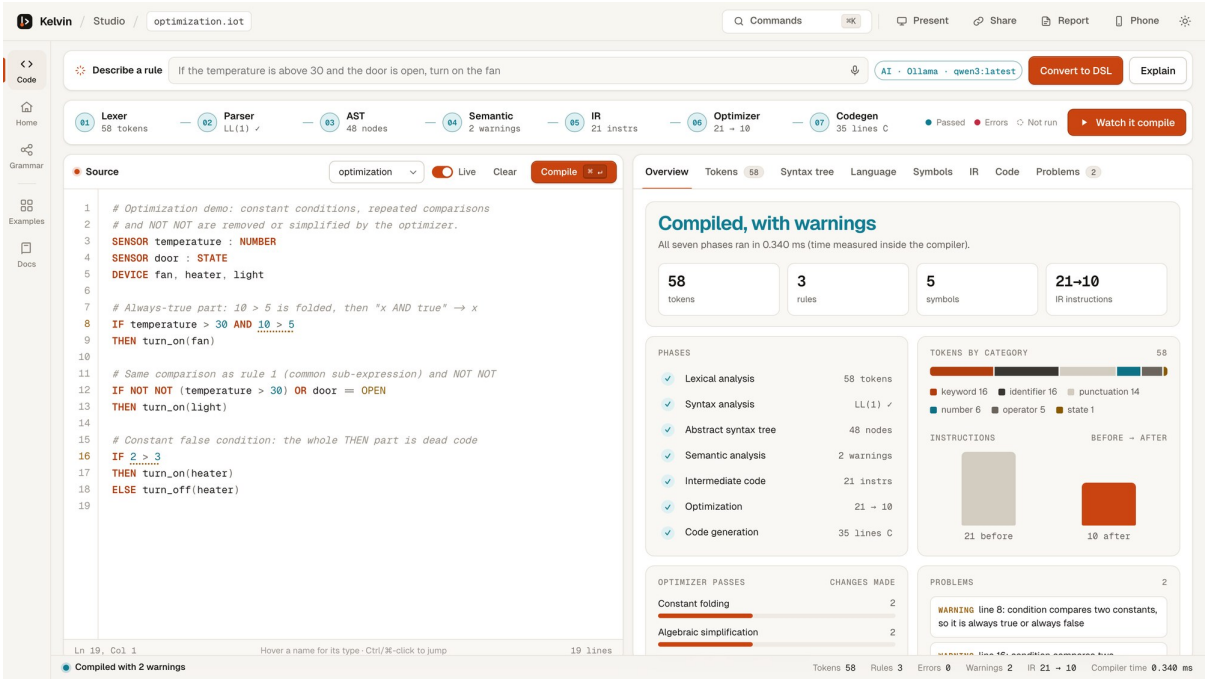


Figure 3: *The Kelvin Studio showing the Overview of a compilation (example 07)*

3.6 Tools and Technologies

Table 2: *Tools and technologies used*

Component	Technology	Notes
Compiler (iotc)	C++17, Apple clang 21 / g++	4,865 lines in 28 source files; no Lex/Yacc or libraries
Build	GNU Make	`make`, `make run`, `make remote`, `make test`
Runtimes	C99, Python 3, Node.js	Identical output format for all targets
Web server	Python 3.14 standard library	`http.server`, JSON API, Server-Sent Events
Front end	HTML, CSS, JavaScript (ES modules)	No frameworks; self-hosted fonts; works offline
AI models	Ollama (Qwen3 8B), Groq, Gemini	Free; offline rule-based fallback
Testing	POSIX shell script, Python unittest	126 compiler checks + 19 unit tests
Version control	Git, GitHub	Private repository
Hardware used	Apple M3, 16 GB RAM, macOS 27	All measurements in Section 4

4. Results and Discussion

4.1 Test Cases

The test suite has two parts. `tests/run_tests.sh` runs 126 checks against the compiler binary, and Python `unittest` runs 19 tests for the server, the phone remote and the AI module. **All 145 tests pass.** Table 3 groups the compiler checks; Table 4 lists the example programs used as end-to-end test cases.

Table 3: *Compiler test checks by area (all passing)*

Area	Checks	What is verified
Examples	7	Each bundled example compiles or fails with the expected message
Lexer and parser	9	Tokens, positions, precedence, panic-mode recovery
Semantic analysis	12	Undeclared names, type mismatches, argument counts, suggestions
IR and optimization	5	Three-address code and instruction counts after optimization
Code generation	6	Generated C compiles with -Werror and gives correct device states
Arithmetic and ELSE IF	11	Arithmetic in conditions and actions; ELSE IF chains
Rule analysis	10	Impossible, always-true, unreachable and conflicting rules
CFG and dataflow	4	Basic blocks, edges, liveness
Generated C for ELSE IF	7	Run-time behaviour of the C output
New examples	16	Security, air quality, kitchen and office programs
Grammar, English, fixes, traces	18	LL(1) table, back-translation, quick fixes, traces
Cross-target equivalence	20	C, Python and JavaScript agree; Arduino sketches compile
JSON	1	Machine-readable output is valid
Python unit tests	19	Remote access rules, run endpoint, AI meaning check, offline converter

Table 4: *Example programs used as test cases*

Program	Purpose	Expected result	Result
01 basic	Problem-statement example	Compiles; fan on when hot and open	Pass
02 smart home	Precedence, NOT, set_level	Compiles, 3 rules	Pass
03 greenhouse	NOT with states, levels	Compiles, 2 rules	Pass
04 lexical error	Bad characters, '=' for '=='	4 lexical errors with fixes	Pass
05 syntax error	Lowercase keyword, missing THEN and ')'	3 syntax errors, recovery	Pass
06 semantic error	Undeclared, type mismatch, wrong arity	6 semantic errors	Pass
07 optimization	Constant conditions, CSE, dead code	21 → 10 instructions, 2 warnings	Pass
08 rule analysis	Impossible, always-true, unreachable, conflict	4 rule warnings with examples	Pass
09 thermostat	ELSE IF chains, arithmetic	Heater level (22 − t) × 10	Pass

Program	Purpose	Expected result	Result
10 home security	Parentheses, ELSE IF, several actions	Alarm on intrusion, dim porch light	Pass
11 air quality	Arithmetic in actions	Vent level (co2 − 800) / 4	Pass
12 kitchen safety	OR conditions, safety shut-off	Gas valve shut, sprinkler on fire	Pass
13 office energy	NOT, long AND chains, arithmetic	Lights and AC follow people and time	Pass

4.2 Sample Outputs

The following outputs are produced by the compiler for the problem-statement example (`examples/01_basic.iot`).

```
# Input program
SENSOR temperature : NUMBER
SENSOR door : STATE
DEVICE fan
IF temperature > 30 AND door == OPEN
THEN turn_on(fan)
```

Phase 1 — tokens (first eight of 23):

```
#    LINE:COL  TOKEN         LEXEME
1    2:1       SENSOR        SENSOR
2    2:8       IDENTIFIER    temperature
3    2:20      COLON         :
4    2:22      TYPE_NUMBER   NUMBER
5    3:1       SENSOR        SENSOR
6    3:8       IDENTIFIER    door
7    3:13      COLON         :
8    3:15      TYPE_STATE    STATE
```

Phases 2–3 — abstract syntax tree (rule part):

```
Rule
├── Condition
│   └── AND
│       ├── Comparison: >
│       │   ├── Identifier: temperature
│       │   └── Number: 30
│       └── Comparison: ==
│           ├── Identifier: door
│           └── State: OPEN
└── Then
    └── Action: turn_on
        └── Identifier: fan
```

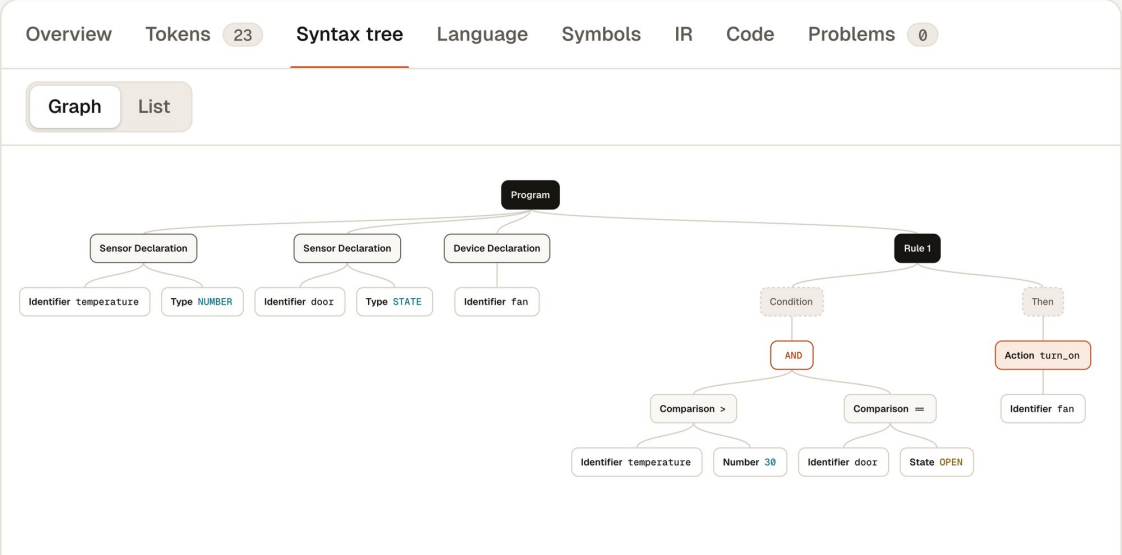


Figure 4: *The same syntax tree drawn by the studio*

Phases 5–6 — intermediate code before and after optimization for `examples/07_optimization.iot`, which contains a constant condition, a repeated comparison, a double negation and a dead branch:

```
BEFORE (21 instructions)                AFTER (10 instructions)
# Rule 1                                # Rule 1
    t1 = temperature > 30                   t1 = temperature > 30
    t2 = 10 > 5                             IF_FALSE t1 GOTO L1
    t3 = t1 AND t2                          CALL turn_on(fan)
    IF_FALSE t3 GOTO L1                 L1:
    CALL turn_on(fan)                   # Rule 2
L1:                                         t7 = door == OPEN
# Rule 2                                    t8 = t1 OR t7
    t4 = temperature > 30                   IF_FALSE t8 GOTO L2
    t5 = NOT t4                             CALL turn_on(light)
    t6 = NOT t5                         L2:
    t7 = door == OPEN                   # Rule 3
    t8 = t6 OR t7                           CALL turn_off(heater)
    IF_FALSE t8 GOTO L2
    CALL turn_on(light)
L2:
# Rule 3
    t9 = 2 > 3
    IF_FALSE t9 GOTO L3
    CALL turn_on(heater)
    GOTO L4
L3:
    CALL turn_off(heater)
L4:
```

Passes that changed the code: constant folding (2), algebraic simplification (2), common sub-expression elimination (1), copy propagation (5), constant-branch elimination (1), unreachable-code elimination (2), jump optimization (3) and dead-code elimination (1).

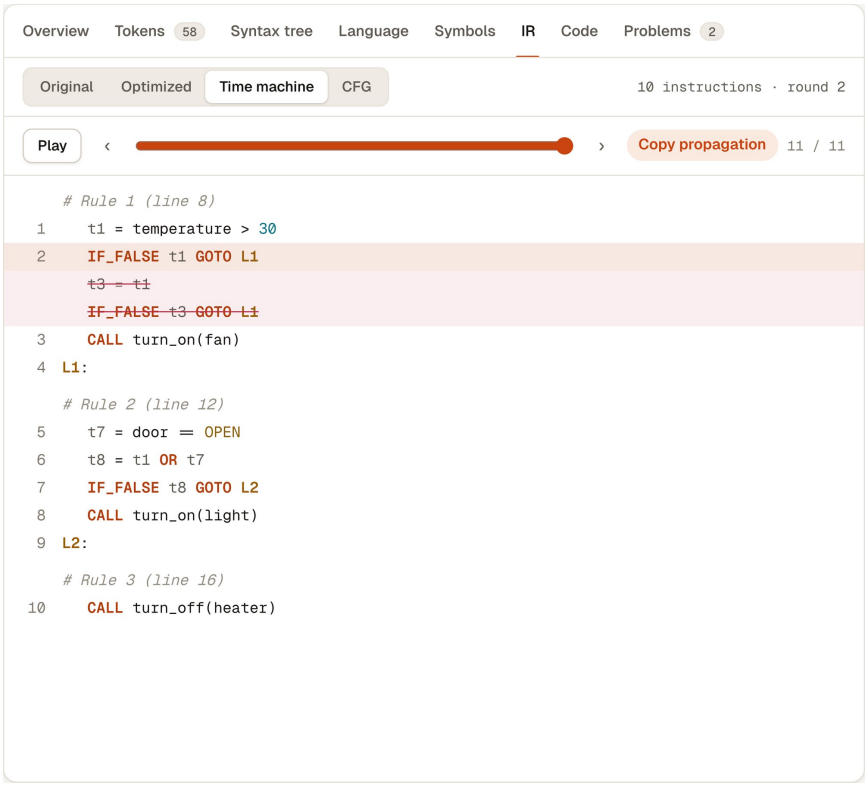


Figure 5: *The optimizer time machine replaying each pass (struck-out lines were removed)*

Phase 7 — generated C for the basic example:

```
void evaluate_rules(void)
{
    /* Read every sensor once, so all rules see the same snapshot */
    double temperature = read_number("temperature");
    state_t door = read_state("door");

    /* Rule 1 (source line 6) */
    bool t1 = temperature > 30;
    bool t2 = door == STATE_OPEN;
    bool t3 = t1 && t2;
    if (t3) {
        turn_on(DEV_FAN);
    }
}
```

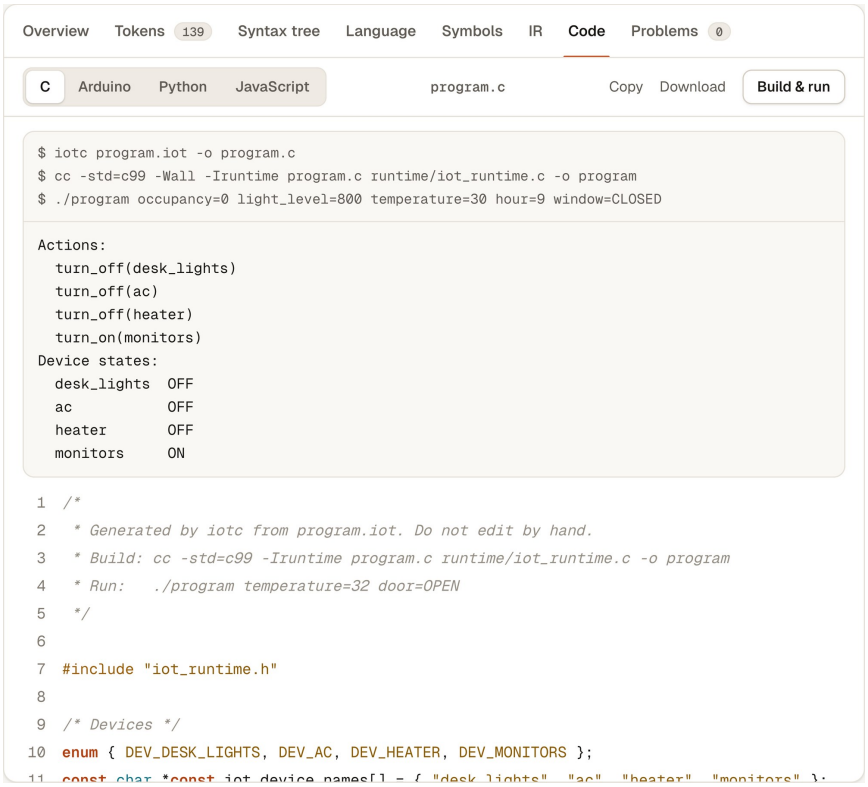


Figure 6: *Build & run: the generated C for example 13 compiled with the system C compiler and executed*

4.3 Error Handling

Each phase reports its own errors with line and column, continues where it safely can, and attaches quick fixes when the correction is unambiguous. Table 5 lists real messages produced for the error examples, and Figures 7 and 8 show them in the studio.

Table 5: *Diagnostics produced by the compiler*

Phase	Message (abridged)	Quick fix
Lexical	unexpected character '@'	Remove '@'
Lexical	unexpected '=': assignment is not supported, did you mean '=='?	Replace '=' with '=='
Syntax	expected 'IF', 'SENSOR' or 'DEVICE' … found 'if' (keywords are uppercase)	Change to 'IF'
Syntax	expected 'THEN' after the condition but found 'turn_on'	Insert 'THEN'
Syntax	expected ',' or ')' in the argument list of 'turn_on' but found end of file	Insert ')'
Semantic	'fan' is not a registered device	Declare 'DEVICE fan'
Semantic	'temprature' is not a declared sensor (did you mean 'temperature'?)	Change to 'temperature'
Semantic	type mismatch: cannot compare STATE sensor 'door' with number 5	—
Semantic	'set_level' expects 2 arguments … but got 1	—
Rule analysis	this condition can never be true, so its actions never run	—
Rule analysis	this ELSE IF can never run: … an earlier condition is already true	—
Rule analysis	conflict: rule 4 turns 'fan' off while rule 3 turns 'fan' on, e.g. when temperature = 31 and door = OPEN	—

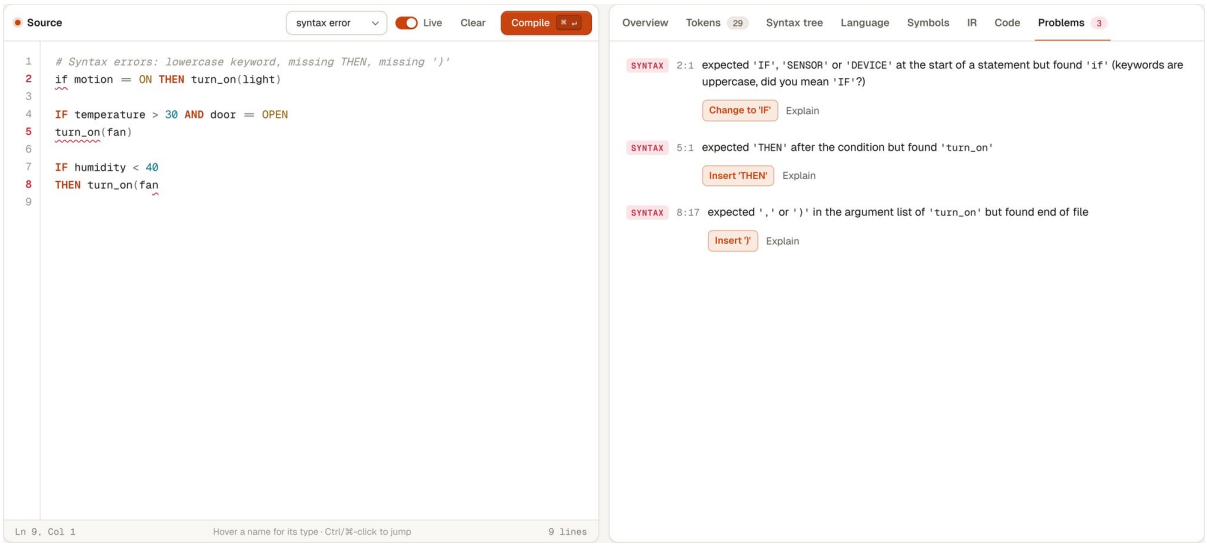


Figure 7: *Syntax errors with panic-mode recovery and one-click fixes (example 05)*

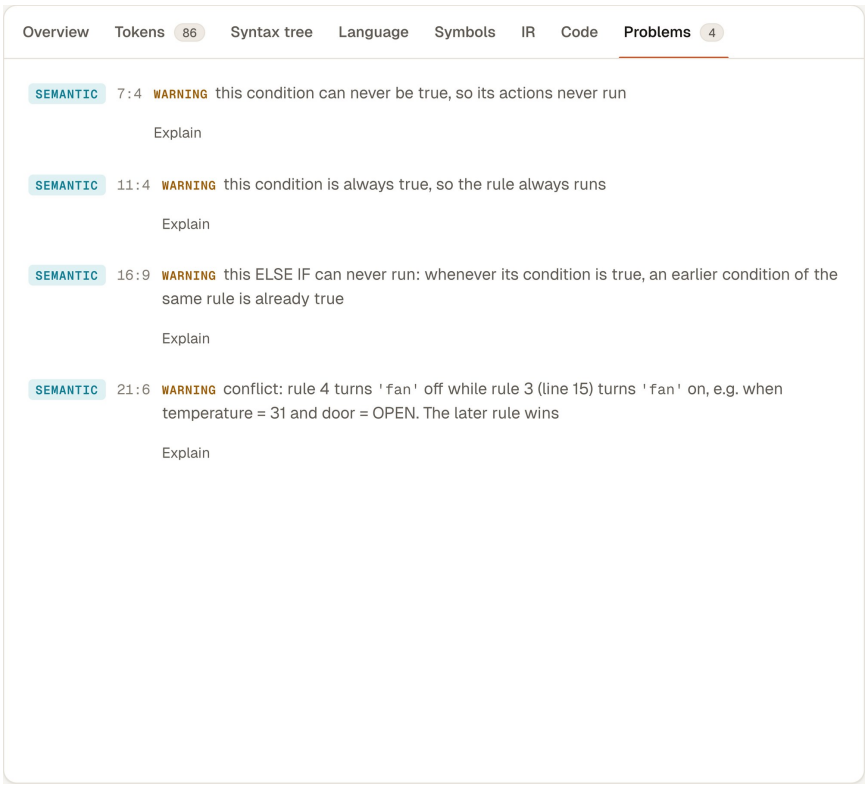


Figure 8: *Rule-analysis warnings with example inputs (example 08)*

4.4 Performance

All measurements were taken on an Apple M3 laptop (16 GB RAM) with the compiler built at `-O2`. *Compiler time* is the time measured inside `iotc` for all seven phases; *end-to-end* includes starting the process and writing the JSON output. Each value is the median of 30 runs.

Table 6: *Compiler performance on the example programs*

Program	Lines	Tokens	Rules	IR before → after	Compiler time (ms)	End-to-end (ms)
basic	7	23	1	6 → 6	0.118	2.46

Program	Lines	Tokens	Rules	IR before → after	Compiler time (ms)	End-to-end (ms)
smart home	20	87	3	27 → 26	0.317	2.83
greenhouse	13	52	2	15 → 15	0.168	2.43
lexical error	4	12	0	— (errors)	0.023	2.00
syntax error	8	29	0	— (errors)	0.045	2.08
semantic error	14	45	3	— (errors)	0.032	2.11
optimization	18	58	3	21 → 10	0.225	2.62
rule analysis	26	86	6	33 → 32	0.388	2.94
thermostat	23	96	2	33 → 33	0.230	2.66
home security	25	102	3	34 → 31	0.325	2.82
air quality	28	118	3	42 → 40	0.365	2.93
kitchen safety	25	98	3	34 → 32	0.342	3.02
office energy	29	139	4	52 → 51	0.567	4.13

The compiler processes every example in under 0.6 ms, and the cost grows roughly linearly with the number of tokens. Programs with errors finish fastest because later phases are skipped. The optimizer removes 52% of the code in the optimization example and 1–3 instructions (mostly repeated comparisons) in realistic programs; programs that are already minimal are left unchanged, as expected.

AI conversion. With the local Qwen3 8B model, converting one sentence took 2.5–2.7 s once the model was loaded (10.4 s for the first request, which includes loading the model). The offline rule-based converter took under 1 ms. In every measured case the final answer compiled without errors.

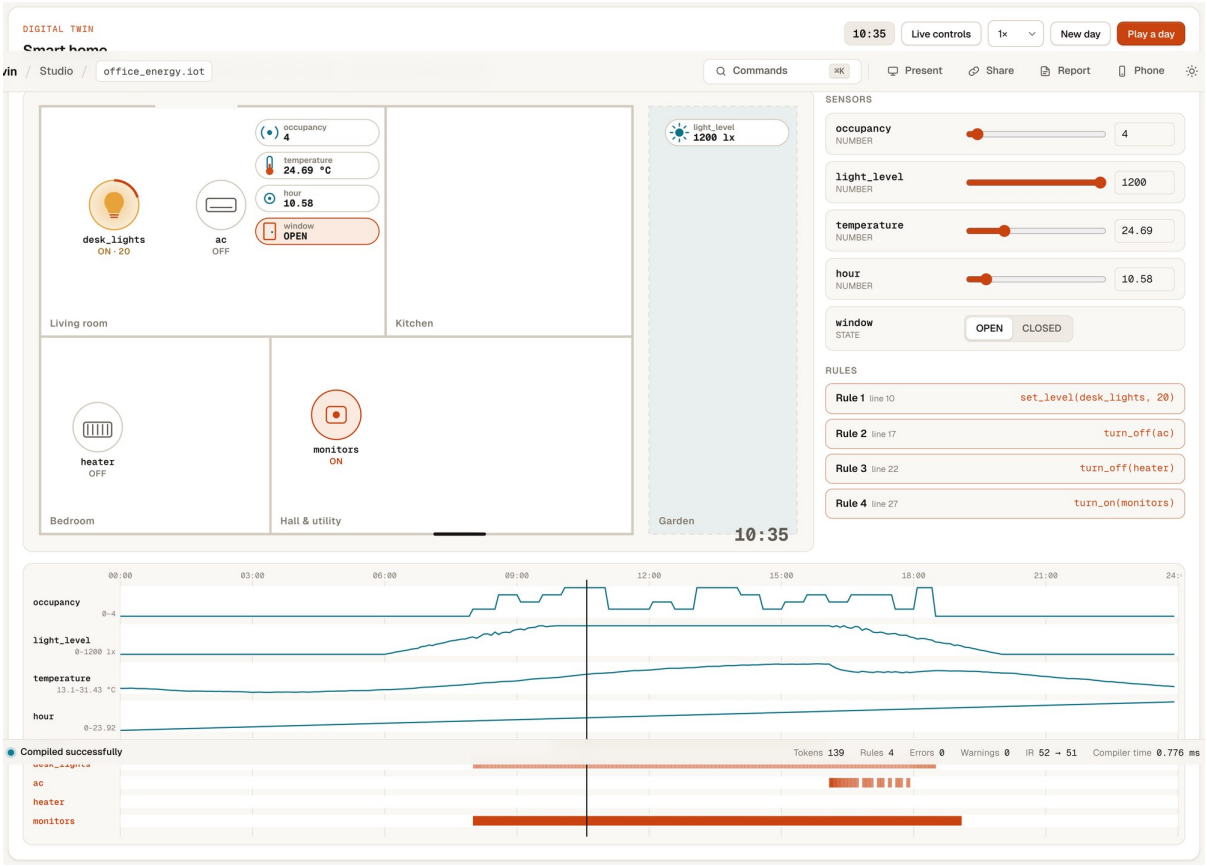


Figure 9: *Digital twin running example 13 during a simulated day (10:55): lights dimmed by daylight, monitors on*

4.5 Novelty

- **Compiler-verified AI:** language-model output is never trusted; it must compile, pass a meaning check and, if needed, survive one compiler-guided repair.
- **Compile-time rule-conflict analysis** with concrete example inputs, which ordinary compilers and trigger–action platforms do not provide.
- **A self-verifying grammar:** the compiler computes FIRST/FOLLOW and the LL(1) table of its own grammar and reports zero conflicts.
- **One IR, four targets** (C, Arduino, Python, JavaScript), with tests proving identical behaviour across targets.
- **Explainable compilation:** the Compiler X-ray replays the real lexer DFA, parser call stack and optimizer passes, and a digital twin runs the optimized IR.

4.6 Limitations

- The language has no timers, schedules, persistent variables or user-defined functions, so rules such as *"turn off the light 10 minutes after motion stops"* cannot be expressed.
- Rule analysis is exact for comparisons against constants but treats arithmetic between sensors conservatively, so some conflicts involving expressions such as `temperature - outside > 5` are not reported.

- The Arduino target generates stub sensor reads that must be connected to real pins; it has been compiled against a stub header but not deployed on hardware.
- Small local language models are slow on a laptop (several seconds per sentence) and can misread complex sentences; the meaning check catches missing numbers and unused sensors but not every misinterpretation.
- The digital twin uses simplified physics and is meant for demonstration, not for engineering-grade simulation.

5. Conclusion and Future Work

This project delivered a complete, working AI-assisted compiler for an IoT automation DSL. All seven phases are implemented by hand in C++17: a DFA lexer, an LL(1) recursive-descent parser with error recovery, an AST with source spans, semantic analysis with satisfiability-based rule analysis, three-address IR, a CFG-based optimizer driven by liveness and available-expressions analysis, and a retargetable back end for four languages. Natural-language rules are drafted by free language models but accepted only after the compiler verifies them. The implementation is supported by 145 passing automated tests, compiles realistic programs in well under a millisecond, and is demonstrated through a web studio, a compiler X-ray and a smart-home digital twin.

Future work.

- Add timers, schedules and state that persists between evaluations, with matching analyses for timing conflicts.
- Extend rule analysis to linear arithmetic between sensors using an SMT solver, and move the IR to SSA form [10].
- Deploy the Arduino/ESP32 output on real hardware with MQTT integration, and add a Home Assistant target.
- Use grammar-constrained decoding [29] so the language model can only produce syntactically valid DSL, and evaluate conversion accuracy on a larger set of English rules.
- Conduct a user study with non-programmers, following the methodology of [15] and [18].

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2011.

[3] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 1998.

[4] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA, USA: Morgan Kaufmann, 1997.

[5] D. E. Knuth, "On the translation of languages from left to right," *Information and Control*, vol. 8, no. 6, pp. 607–639, 1965.

[6] T. J. Parr and R. W. Quong, "ANTLR: A predicated-LL(k) parser generator," *Software: Practice and Experience*, vol. 25, no. 7, pp. 789–810, 1995.

[7] T. Parr and K. Fisher, "LL(*): The foundation of the ANTLR parser generator," in* Proc. 32nd ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2011, pp. 425–436.

[8] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 1–19, 1970.

[9] G. A. Kildall, "A unified approach to global program optimization," in *Proc. 1st ACM Symp. Principles of Programming Languages (POPL)*, 1973, pp. 194–206.

[10] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Programming Languages and Systems*, vol. 13, no. 4, pp. 451–490, 1991.

[11] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[12] M. Mernik, J. Heering, and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Computing Surveys*, vol. 37, no. 4, pp. 316–344, 2005.

[13] A. van Deursen, P. Klint, and J. Visser, "Domain-specific languages: An annotated bibliography," *ACM SIGPLAN Notices*, vol. 35, no. 6, pp. 26–36, 2000.

[14] M. Fowler, *Domain-Specific Languages*. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[15] B. Ur, E. McManus, M. Pak Yong Ho, and M. L. Littman, "Practical trigger-action programming in the smart home," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, 2014, pp. 803–812.

[16] B. Ur, M. Pak Yong Ho, S. Brawner, J. Lee, S. Mennicken, N. Picard, D. Schulze, and M. L. Littman, "Trigger-action programming in the wild: An analysis of 200,000 IFTTT recipes," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2016, pp. 3227–3231.

[17] J. Huang and M. Cakmak, "Supporting mental model accuracy in trigger-action programming," in *Proc. ACM Int. Joint Conf. Pervasive and Ubiquitous Computing (UbiComp)*, 2015, pp. 215–225.

[18] W. Brackenbury, A. Deora, J. Ritchey, J. Vallee, W. He, G. Wang, M. L. Littman, and B. Ur, "How users interpret bugs in trigger-action programming," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2019, pp. 1–12.

[19] Z. B. Celik, P. McDaniel, and G. Tan, "Soteria: Automated IoT safety and security analysis," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2018, pp. 147–158.

[20] Z. B. Celik, G. Tan, and P. McDaniel, "IoTGuard: Dynamic enforcement of security and safety policy in commodity IoT," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2019.

[21] W. Ding and H. Hu, "On the safety of IoT device physical interaction control," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 832–846.

[22] Q. Wang, P. Datta, W. Yang, S. Liu, A. Bates, and C. A. Gunter, "Charting the attack surface of trigger-action IoT platforms," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2019, pp. 1439–1453.

[23] A. Desai, S. Gulwani, V. Hingorani, N. Jain, A. Karkare, M. Marron, S. R, and S. Roy, "Program synthesis using natural language," in *Proc. 38th Int. Conf. Software Engineering (ICSE)*, 2016, pp. 345–356.

[24] X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst, "NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system," in *Proc. 11th Int. Conf. Language Resources and Evaluation (LREC)*, 2018.

[25] P. Yin and G. Neubig, "A syntactic neural model for general-purpose code generation," in *Proc. 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017, pp. 440–450.

[26] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[27] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[28] J. Austin *et al.*, "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[29] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, "Synchromesh: Reliable code generation from pre-trained language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[30] A. Yang *et al.*, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.